PYTHON ACTIVITY-BASED LEARNING REPORT

Name: Azeez	**Roll No:** 26000145
Branch: CSE	**Course:** Python Programming

Selected 10 Beginner-Friendly Python Case Studies

1. ATM Transaction Simulation (Case Study 13)
2. Examination Result Calculator (Case Study 11)
3. Student Attendance Calculator (Case Study 12)
4. Student Performance Analysis (Case Study 17)
5. Number Pattern Generator (Case Study 27)
6. Password Validation (Case Study 29)
7. Examination Answer Evaluation (Case Study 31)
8. Text Compression (Case Study 35)
9. Examination Eligibility System (Case Study 39)
10. Food Waste Monitoring (Case Study 53)

Department of Computer Science and Engineering

Case 1: ATM Transaction Simulation (Case Study 13)

Problem Statement

Write a Python program to perform a simple ATM transaction. The user can deposit or withdraw money, and the program displays the updated account balance. A withdrawal must not exceed the available balance.

Algorithm

• Start the program.

• Set the initial balance to Rs. 5000.

• Display the deposit and withdrawal options.

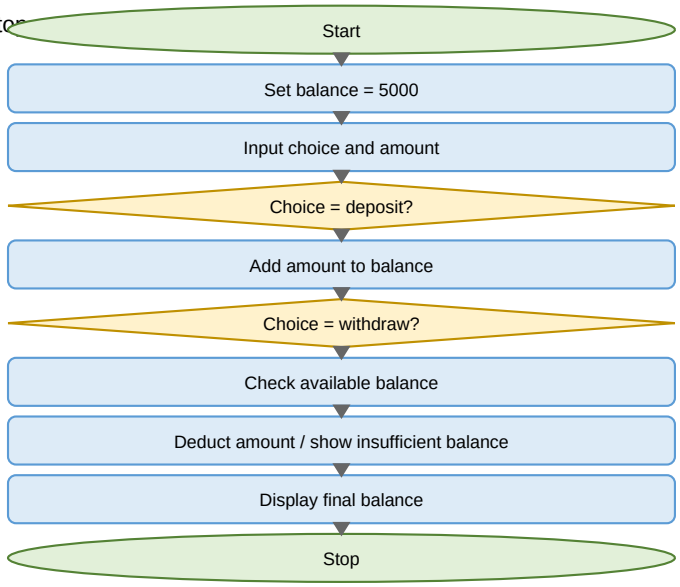
• Read the user's choice and transaction amount.

• If the choice is deposit, add the amount to the balance.

• If the choice is withdraw, check whether the amount is available. Deduct it if sufficient; otherwise display an insufficient-balance message.

• Display the final balance and stop.

Flowchart



Pseudocode

```
START
    balance = 5000
    DISPLAY deposit and withdraw options
    INPUT choice
    INPUT amount
    IF choice = 1 THEN
        balance = balance + amount
        DISPLAY "Amount Deposited"
    ELSE IF choice = 2 THEN
        IF amount <= balance THEN
            balance = balance - amount
            DISPLAY "Amount Withdrawn"
        ELSE
            DISPLAY "Insufficient Balance"
        END IF
    ELSE
        DISPLAY "Invalid Choice"
    END IF
    DISPLAY balance
STOP
```

Source Code

```
balance = 5000

print("1. Deposit")
print("2. Withdraw")

choice = int(input("Enter your choice: "))
amount = int(input("Enter amount: "))

if choice == 1:
    balance = balance + amount
    print("Amount Deposited")
elif choice == 2:
    if amount <= balance:
        balance = balance - amount
        print("Amount Withdrawn")
```

```
    else:
        print("Insufficient Balance")
else:
    print("Invalid Choice")

print("Final Balance:", balance)
```

Sample Input

```
Enter your choice: 1\nEnter amount: 1000
```

Sample Output

```
1. Deposit\n2. Withdraw\nAmount Deposited\nFinal Balance: 6000
```

Conclusion

The program successfully performs the required task using basic Python statements and displays the expected result.

Case 2: Examination Result Calculator (Case Study 11)

Problem Statement

Write a Python program to read marks in three subjects, calculate the total and average, and display PASS if the student scores at least 35 marks in every subject. Otherwise, display FAIL.

Algorithm

• Start the program.

• Read marks in three subjects.

• Calculate the total marks by adding the three marks.

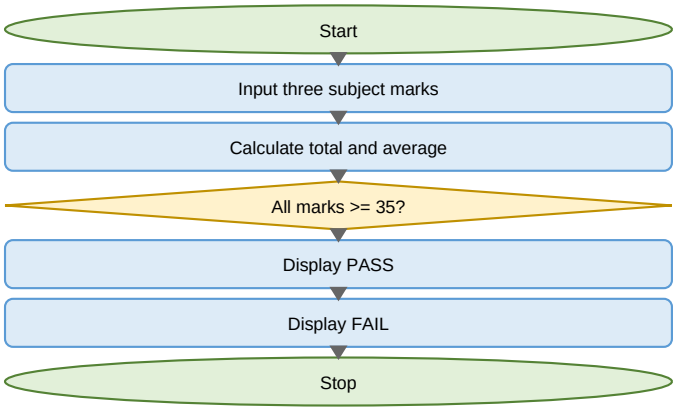
• Calculate the average by dividing the total by 3.

• If all three subject marks are at least 35, display PASS.

• Otherwise, display FAIL.

• Display total and average, then stop.

Flowchart



Pseudocode

```
START
    INPUT m1, m2, m3
    total = m1 + m2 + m3
    average = total / 3
    DISPLAY total and average
    IF m1 >= 35 AND m2 >= 35 AND m3 >= 35 THEN
        DISPLAY "PASS"
    ELSE
        DISPLAY "FAIL"
    END IF
STOP
```

Source Code

```
m1 = int(input("Enter subject 1 marks: "))
m2 = int(input("Enter subject 2 marks: "))
m3 = int(input("Enter subject 3 marks: "))

total = m1 + m2 + m3
average = total / 3

print("Total Marks:", total)
print("Average:", average)

if m1 >= 35 and m2 >= 35 and m3 >= 35:
    print("Result: PASS")
else:
    print("Result: FAIL")
```

Sample Input

```
Enter subject 1 marks: 80\nEnter subject 2 marks: 75\nEnter subject 3 marks: 90
```

Sample Output

```
Total Marks: 245\nAverage: 81.66666666666667\nResult: PASS
```

Conclusion

The program successfully performs the required task using basic Python statements and displays the expected result.

Case 3: Student Attendance Calculator (Case Study 12)

Problem Statement

Write a Python program to calculate a student's attendance percentage using the total number of classes and the number of classes attended. The student is eligible for the examination if attendance is at least 75%.

Algorithm

• Start the program.

• Read total classes and attended classes.

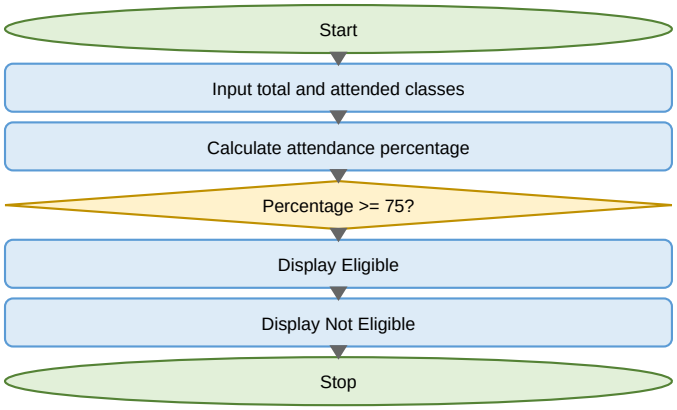
• Calculate attendance percentage as attended classes divided by total classes multiplied by 100.

• If attendance is at least 75%, display Eligible for Exam.

• Otherwise, display Not Eligible for Exam.

• Display the attendance percentage and stop.

Flowchart



Pseudocode

```
START
    INPUT total, attended
    percentage = (attended / total) * 100
    DISPLAY percentage
    IF percentage >= 75 THEN
        DISPLAY "Eligible for Exam"
    ELSE
        DISPLAY "Not Eligible for Exam"
    END IF
STOP
```

Source Code

```
total = int(input("Enter total classes: "))
attended = int(input("Enter attended classes: "))

percentage = (attended / total) * 100

print("Attendance:", percentage, "%")

if percentage >= 75:
    print("Eligible for Exam")
else:
    print("Not Eligible for Exam")
```

Sample Input

```
Enter total classes: 40\nEnter attended classes: 32
```

Sample Output

```
Attendance: 80.0 %\nEligible for Exam
```

Conclusion

The program successfully performs the required task using basic Python statements and displays the expected result.

Case 4: Student Performance Analysis (Case Study 17)

Problem Statement

Write a Python program to accept marks in a number of subjects, calculate the total and average marks, and classify performance as Excellent, Good, Average, or Poor based on the average.

Algorithm

• Start the program.

• Read the number of subjects.

• Create an empty list to store marks.

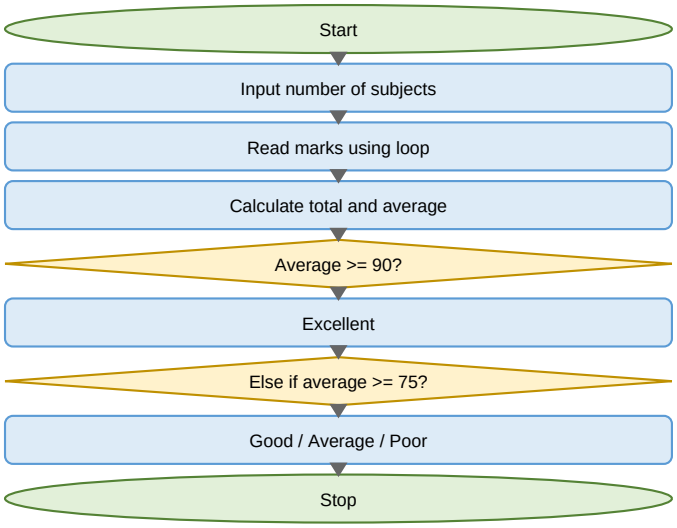
• Use a loop to read each subject mark and add it to the list.

• Calculate total marks and average marks.

• If average is 90 or above, display Excellent; else if 75 or above, display Good; else if 50 or above, display Average; otherwise display Poor.

• Stop the program.

Flowchart



Pseudocode

```
START
    INPUT n
    CREATE empty marks list
    FOR each subject from 1 to n
        INPUT mark
        ADD mark to marks
    END FOR
    total = SUM(marks)
    average = total / n
    DISPLAY total and average
    IF average >= 90 THEN DISPLAY "Excellent"
    ELSE IF average >= 75 THEN DISPLAY "Good"
    ELSE IF average >= 50 THEN DISPLAY "Average"
    ELSE DISPLAY "Poor"
    END IF
STOP
```

Source Code

```
marks = []

n = int(input("Enter number of subjects: "))

for i in range(n):
    m = int(input("Enter marks: "))
    marks.append(m)

total = sum(marks)
average = total / n

print("Total Marks:", total)
print("Average Marks:", average)

if average >= 90:
    print("Excellent")
elif average >= 75:
    print("Good")
elif average >= 50:
```

```
    print("Average")
else:
    print("Poor")
```

Sample Input

```
Enter number of subjects: 3\nEnter marks: 90\nEnter marks: 80\nEnter marks: 85
```

Sample Output

```
Total Marks: 255\nAverage Marks: 85.0\nGood
```

Conclusion

The program successfully performs the required task using basic Python statements and displays the expected result.

Case 5: Number Pattern Generator (Case Study 27)

Problem Statement

Write a Python program to print an increasing number pattern. The user enters the number of rows, and each row displays numbers from 1 up to the row number.

Algorithm

• Start the program.

• Read the number of rows.

• Use an outer loop from 1 to the number of rows.

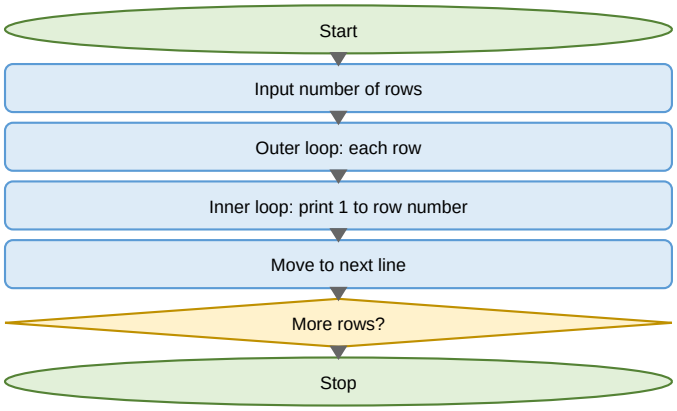
• Inside it, use an inner loop from 1 to the current row number.

• Print the inner-loop number on the same line.

• After each row, move to the next line.

• Stop the program.

Flowchart



Pseudocode

```
START
    INPUT n
    FOR i = 1 TO n
        FOR j = 1 TO i
            DISPLAY j on the same line
        END FOR
        MOVE to next line
    END FOR
STOP
```

Source Code

```
n = int(input("Enter number of rows: "))

for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

Sample Input

```
Enter number of rows: 4
```

Sample Output

```
1\n1 2\n1 2 3\n1 2 3 4
```

Conclusion

The program successfully performs the required task using basic Python statements and displays the expected result.

Case 6: Password Validation (Case Study 29)

Problem Statement

Write a Python program to check whether a password contains at least 8 characters. If the length is 8 or more, display Valid Password; otherwise, display Invalid Password.

Algorithm

• Start the program.

• Read a password from the user.

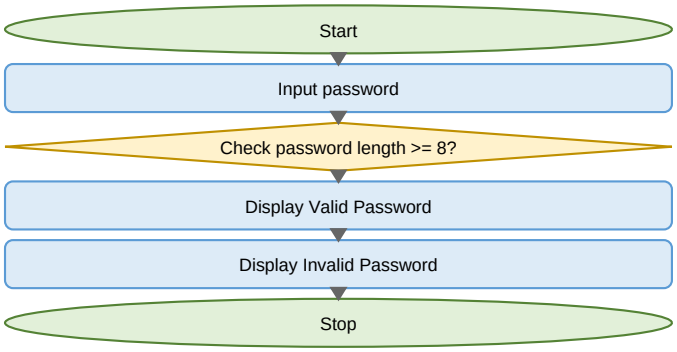
• Find the number of characters using the length function.

• If the length is at least 8, display Valid Password.

• Otherwise, display Invalid Password and the minimum-length requirement.

• Stop the program.

Flowchart



Pseudocode

```
START
    INPUT password
    IF LENGTH(password) >= 8 THEN
        DISPLAY "Valid Password"
    ELSE
        DISPLAY "Invalid Password"
        DISPLAY "Password must contain 8 characters"
    END IF
STOP
```

Source Code

```
password = input("Enter password: ")

if len(password) >= 8:
    print("Valid Password")
else:
    print("Invalid Password")
    print("Password must contain 8 characters")
```

Sample Input

```
Enter password: Azeez123
```

Sample Output

```
Valid Password
```

Conclusion

The program successfully performs the required task using basic Python statements and displays the expected result.

Case 7: Examination Answer Evaluation (Case Study 31)

Problem Statement

Write a Python program to compare five answers entered by a student with a fixed answer key. Award 2 marks for every correct answer and display the number of correct answers and total marks.

Algorithm

• Start the program.

• Create an answer key containing five correct answers.

• Set the score to zero.

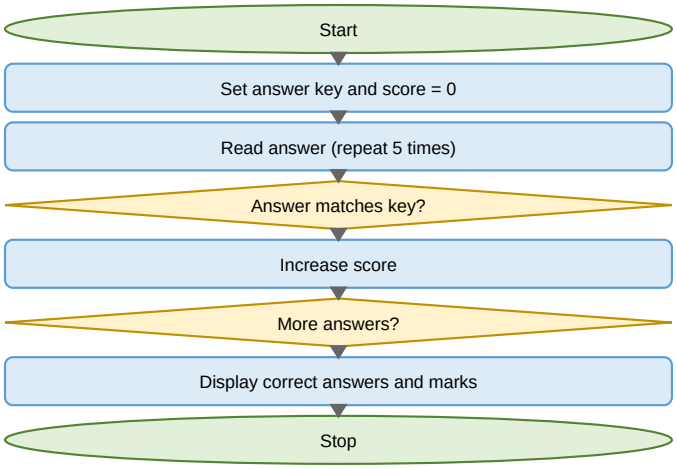
• Repeat five times: read an answer and compare it with the matching answer-key item.

• If the answer is correct, increase the score by 1.

• Display the number of correct answers and total marks (score multiplied by 2).

• Stop the program.

Flowchart



Pseudocode

```
START
    key = [A, B, C, D, A]
    score = 0
    FOR i = 0 TO 4
        INPUT answer
        IF UPPER(answer) = key[i] THEN
            score = score + 1
        END IF
    END FOR
    DISPLAY score
    DISPLAY score * 2
STOP
```

Source Code

```
key = ["A", "B", "C", "D", "A"]
score = 0

for i in range(5):
    answer = input("Enter answer: ")

    if answer.upper() == key[i]:
        score = score + 1

print("Total Correct Answers:", score)
print("Total Marks:", score * 2)
```

Sample Input

```
Enter answer: A\nEnter answer: B\nEnter answer: C\nEnter answer: D\nEnter answer: B
```

Sample Output

```
Total Correct Answers: 4\nTotal Marks: 8
```

Conclusion

The program successfully performs the required task using basic Python statements and displays the expected result.

Case 8: Text Compression (Case Study 35)

Problem Statement

Write a Python program to compress a string by counting consecutive repeated characters. For example, the string 'aaabbc' is compressed as 'a3b2c1'.

Algorithm

• Start the program.

• Read a string from the user.

• Set result to an empty string and count to 1.

• Compare each character with the previous character.

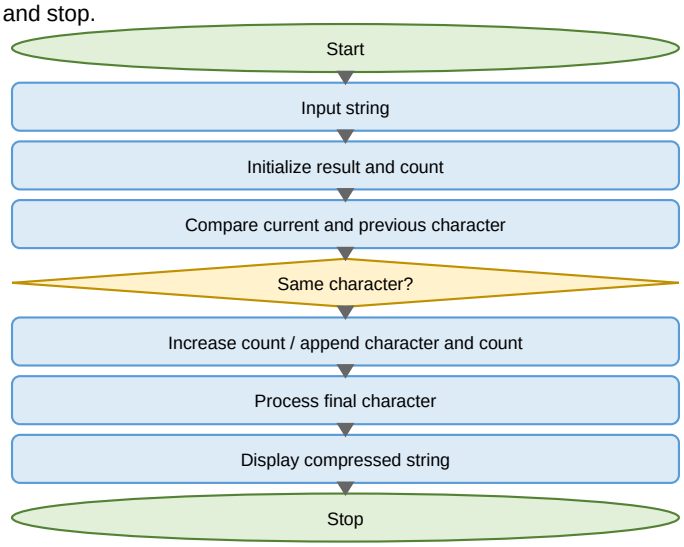
• If both characters are equal, increase count by 1.

• Otherwise, add the previous character and its count to the result, then reset count to 1.

• After the loop, add the final character and its count.

• Display the compressed string and stop.

Flowchart



Pseudocode

```
START
    INPUT text
    result = ""
    count = 1
    FOR i = 1 TO LENGTH(text)-1
        IF text[i] = text[i-1] THEN
            count = count + 1
        ELSE
            ADD text[i-1] and count to result
            count = 1
        END IF
    END FOR
    IF text is not empty THEN
        ADD last character and count to result
    END IF
    DISPLAY result
STOP
```

Source Code

```
text = input("Enter a string: ")

result = ""
count = 1

for i in range(1, len(text)):
    if text[i] == text[i - 1]:
        count = count + 1
    else:
        result = result + text[i - 1] + str(count)
        count = 1

if len(text) > 0:
    result = result + text[-1] + str(count)

print("Compressed String:", result)
```

Sample Input

```
Enter a string: aaabbc
```

Sample Output

```
Compressed String: a3b2c1
```

Conclusion

The program successfully performs the required task using basic Python statements and displays the expected result.

Case 9: Examination Eligibility System (Case Study 39)

Problem Statement

Write a Python program to determine examination eligibility using four conditions: attendance must be at least 75%, laboratory work must be completed, internal marks must be at least 40, and fees must be paid.

Algorithm

• Start the program.

• Read attendance percentage, laboratory completion status, internal marks, and fee-payment status.

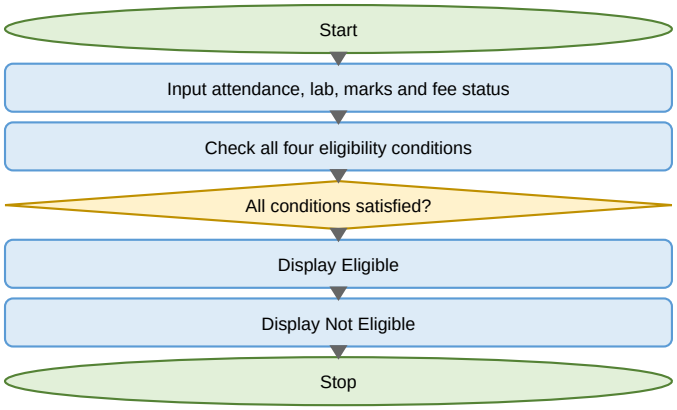
• Check whether attendance is at least 75%, lab status is yes, internal marks are at least 40, and fee status is yes.

• If all conditions are satisfied, display Eligible for Examination.

• Otherwise, display Not Eligible for Examination.

• Stop the program.

Flowchart



Pseudocode

```
START
    INPUT attendance, lab, marks, fees
    IF attendance >= 75 AND lab = "yes"
       AND marks >= 40 AND fees = "yes" THEN
        DISPLAY "Eligible for Examination"
    ELSE
        DISPLAY "Not Eligible for Examination"
    END IF
STOP
```

Source Code

```
attendance = int(input("Enter attendance percentage: "))
lab = input("Lab completed? (yes/no): ").lower()
marks = int(input("Enter internal marks: "))
fees = input("Fees paid? (yes/no): ").lower()

if attendance >= 75 and lab == "yes" and marks >= 40 and fees == "yes":
    print("Eligible for Examination")
else:
    print("Not Eligible for Examination")
```

Sample Input

```
Enter attendance percentage: 80\nLab completed? (yes/no): yes\nEnter internal marks: 65\nFees paid? (yes/no): yes
```

Sample Output

```
Eligible for Examination
```

Conclusion

The program successfully performs the required task using basic Python statements and displays the expected result.

Case 10: Food Waste Monitoring (Case Study 53)

Problem Statement

Write a Python program to calculate the quantity and percentage of food wasted. If food wastage is greater than 20% of the meals prepared, display High Food Wastage; otherwise, display Food Wastage is Normal.

Algorithm

• Start the program.

• Read the number of meals prepared and meals served.

• Calculate food wasted by subtracting meals served from meals prepared.

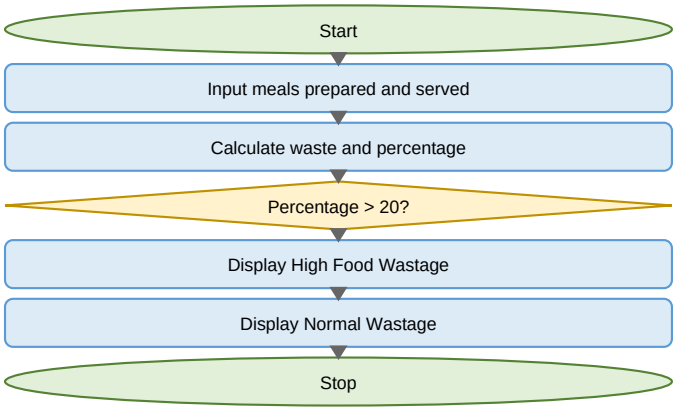
• Calculate wastage percentage as food wasted divided by meals prepared multiplied by 100.

• If the percentage is greater than 20%, display High Food Wastage.

• Otherwise, display Food Wastage is Normal.

• Display the quantity and percentage wasted, then stop.

Flowchart



Pseudocode

```
START
    INPUT prepared, served
    waste = prepared - served
    percentage = (waste / prepared) * 100
    DISPLAY waste and percentage
    IF percentage > 20 THEN
        DISPLAY "High Food Wastage"
    ELSE
        DISPLAY "Food Wastage is Normal"
    END IF
STOP
```

Source Code

```
prepared = int(input("Enter meals prepared: "))
served = int(input("Enter meals served: "))

waste = prepared - served
percentage = (waste / prepared) * 100

print("Food Wasted:", waste)
print("Waste Percentage:", percentage, "%")

if percentage > 20:
    print("High Food Wastage")
else:
    print("Food Wastage is Normal")
```

Sample Input

```
Enter meals prepared: 100\nEnter meals served: 75
```

Sample Output

```
Food Wasted: 25\nWaste Percentage: 25.0 %\nHigh Food Wastage
```

Conclusion

The program successfully performs the required task using basic Python statements and displays the expected result.